Test Case 1

University Scholarship Allocation System in Python

1. Problem Statement

Develop a menu-driven Python program to allocate university scholarships based on students' aggregate marks, subject-wise minimum marks, attendance percentage, and family income. The program identifies eligible students, ranks them according to aggregate marks, and resolves ties using attendance percentage and lower family income. Finally, it displays the ranked list of eligible students.

Eligibility criteria (example):

- Aggregate marks: at least 75%.
- Minimum score in every subject: 60 marks out of 100.
- Attendance: at least 75%.
- Annual family income: not more than ₹2,50,000.
- Tie-breaking: higher aggregate marks, then higher attendance, then lower family income.

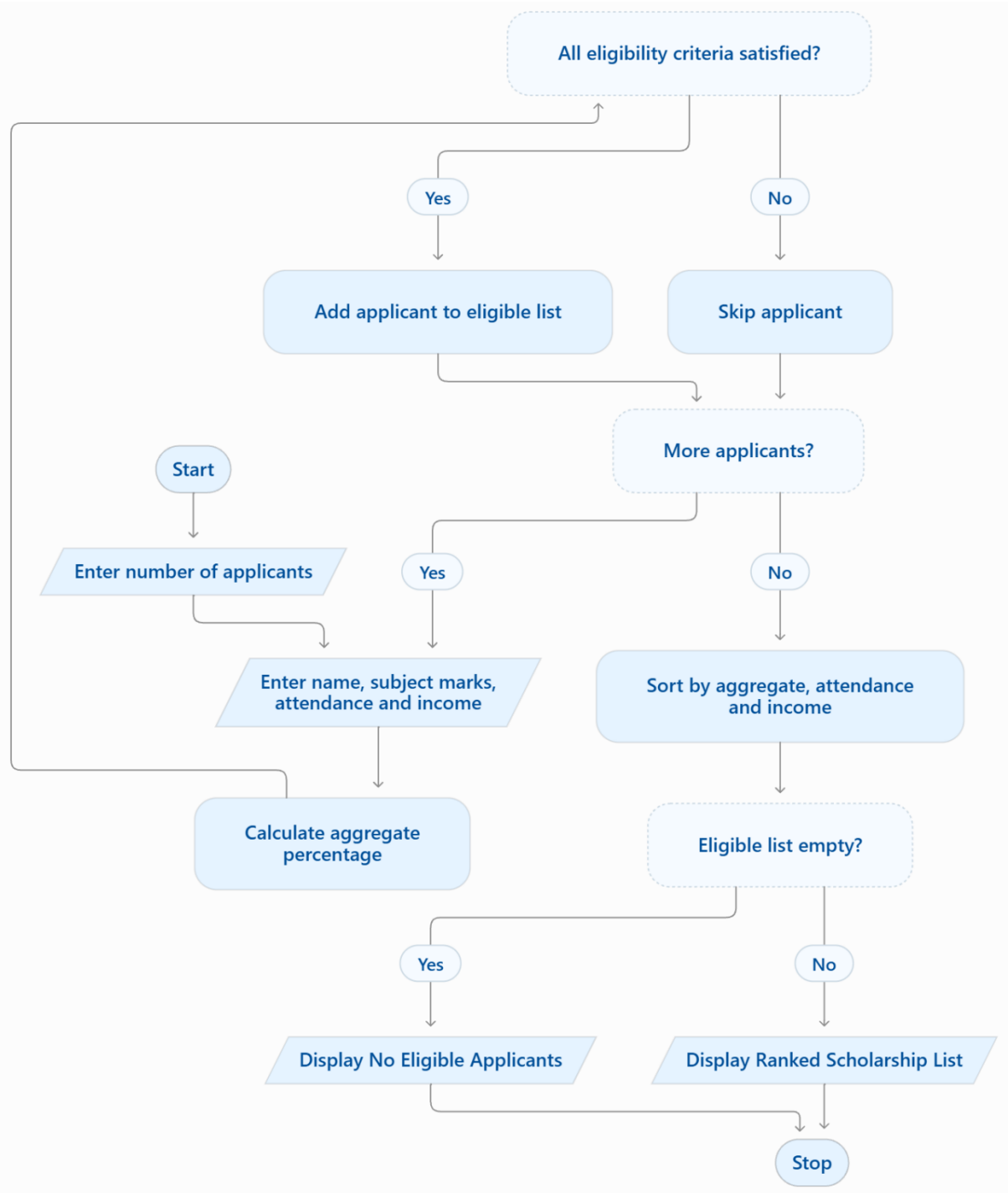
These are sample rules for the program and can be changed according to university policy.

2. Algorithm

1. Start the program.
2. Read the number of scholarship applicants.
3. For each applicant, input the name, subject marks, attendance, and annual family income.
4. Calculate aggregate percentage from the subject marks.
5. Check all scholarship eligibility conditions.
6. Store eligible applicants in a list.
7. Sort eligible applicants by aggregate percentage in descending order.
8. If aggregate marks are equal, rank higher attendance first.
9. If both are equal, rank lower family income first.

10. Display the ranked list of eligible students.
11. If no students qualify, display an appropriate message.
12. Stop the program.

3. Flowchart



4.source code

```
n = int(input("Number of applicants: "))
eligible = []

for i in range(n):
    name = input("\nStudent name: ")
    marks = tuple(map(int, input("Enter 3 subject marks: ").split()))
    attendance = float(input("Attendance (%): "))
    income = int(input("Annual family income: ₹"))
    aggregate = sum(marks) / len(marks)

    if aggregate >= 75 and min(marks) >= 60 and attendance >= 75 and income <=
250000:
        eligible.append((name, aggregate, attendance, income))

eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))

if eligible:
    print("\nScholarship Ranking")
    for rank, (name, marks, att, inc) in enumerate(eligible, 1):
        print(rank, name, "Aggregate:", marks,
              "Attendance:", att, "Income:", inc)
else:
    print("No eligible applicants found.")
```

5. Sample Input

```
Number of applicants: 3

Student name: Ravi
Enter 3 subject marks: 90 80 85
Attendance (%): 90
Annual family income: ₹200000

Student name: Priya
Enter 3 subject marks: 90 80 85
Attendance (%): 95
Annual family income: ₹220000

Student name: Aman
Enter 3 subject marks: 70 80 75
Attendance (%): 80
Annual family income: ₹250000
```

6. Sample Output

```
Scholarship Ranking
1 Priya Aggregate: 85.0 Attendance: 95.0 Income: 220000
2 Ravi Aggregate: 85.0 Attendance: 90.0 Income: 200000
3 Aman Aggregate: 75.0 Attendance: 80.0 Income: 250000
```

Test Case 2

Progressive Electricity Billing System in Python

1. Problem Statement

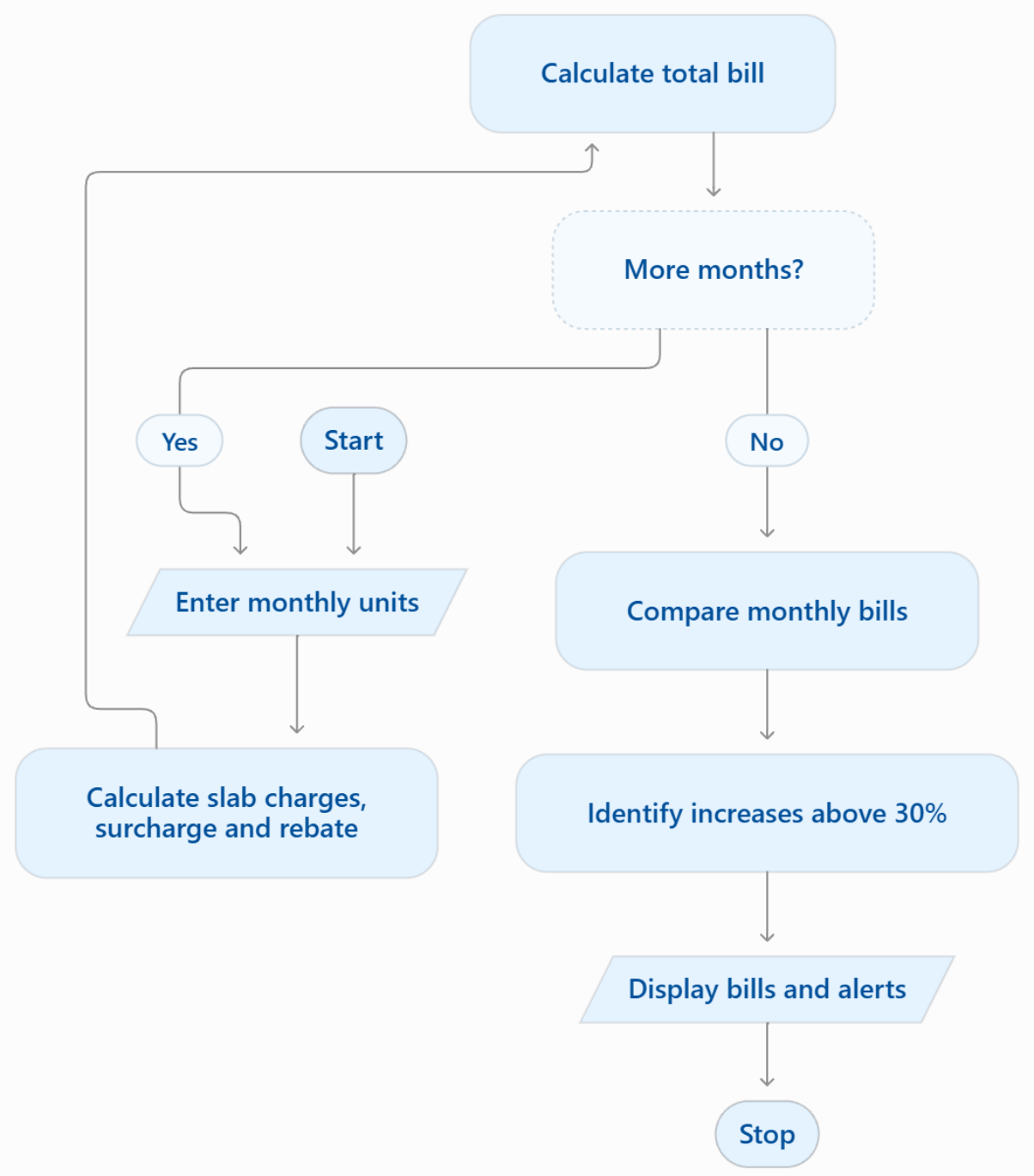
Progressive Electricity Billing: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

2. Algorithm

1. Start the program.
2. Input the number of months.
3. For each month, enter electricity consumption in units.
4. Calculate energy charges using progressive slabs.

5. Add fixed charges and surcharge.
6. Apply the rebate if eligible.
7. Compare the current bill with the previous month's bill.
8. Identify increases greater than 30%.
9. Display monthly bills and unusual increases.
10. Stop.

3. Flowchart



4.source code

```
n = int(input("Enter number of months: "))
previous = 0

for month in range(1, n + 1):
    units = int(input(f"Month {month} units: "))

    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 200 + (units - 100) * 3
    else:
        energy = 500 + (units - 200) * 5

    bill = energy + 50 + energy * 0.05
    if units < 100:
        bill -= energy * 0.10

    print(f"Month {month} bill: ₹{bill:.2f}")
    if previous > 0 and bill > previous * 1.30:
        print("Unusual increase detected!")
    previous = bill

print("Billing completed.")
```

5. Sample Input

```
Enter number of months: 3
Month 1 units: 80
Month 2 units: 120
Month 3 units: 250
```

6. Sample Output

```
Month 1 bill: ₹201.60
Month 2 bill: ₹365.00
Unusual increase detected!
Month 3 bill: ₹1062.50
Unusual increase detected!
Billing completed.
```

Test case 3

Hospital Billing and Patient Classification in Python

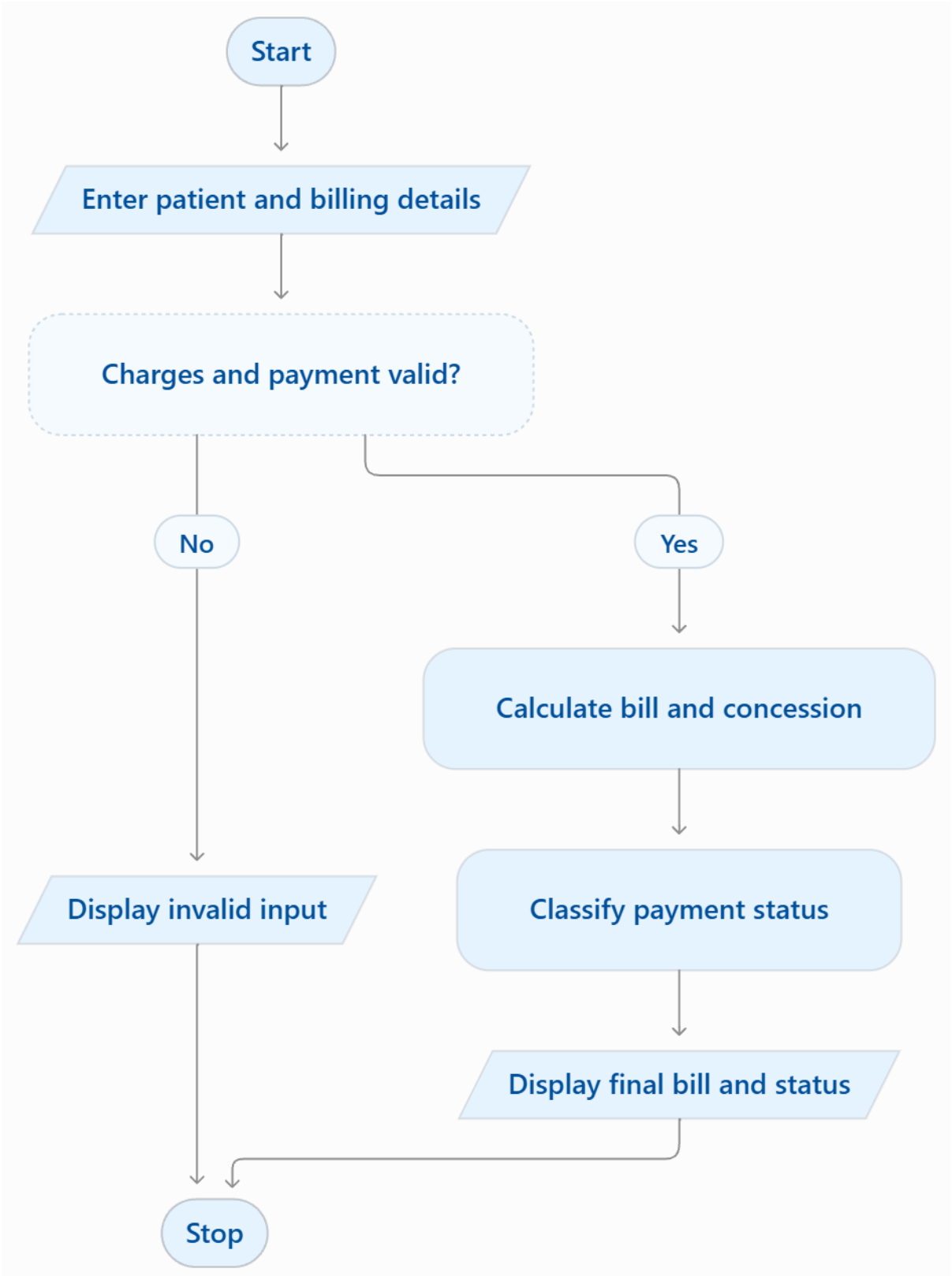
1. Problem Statement

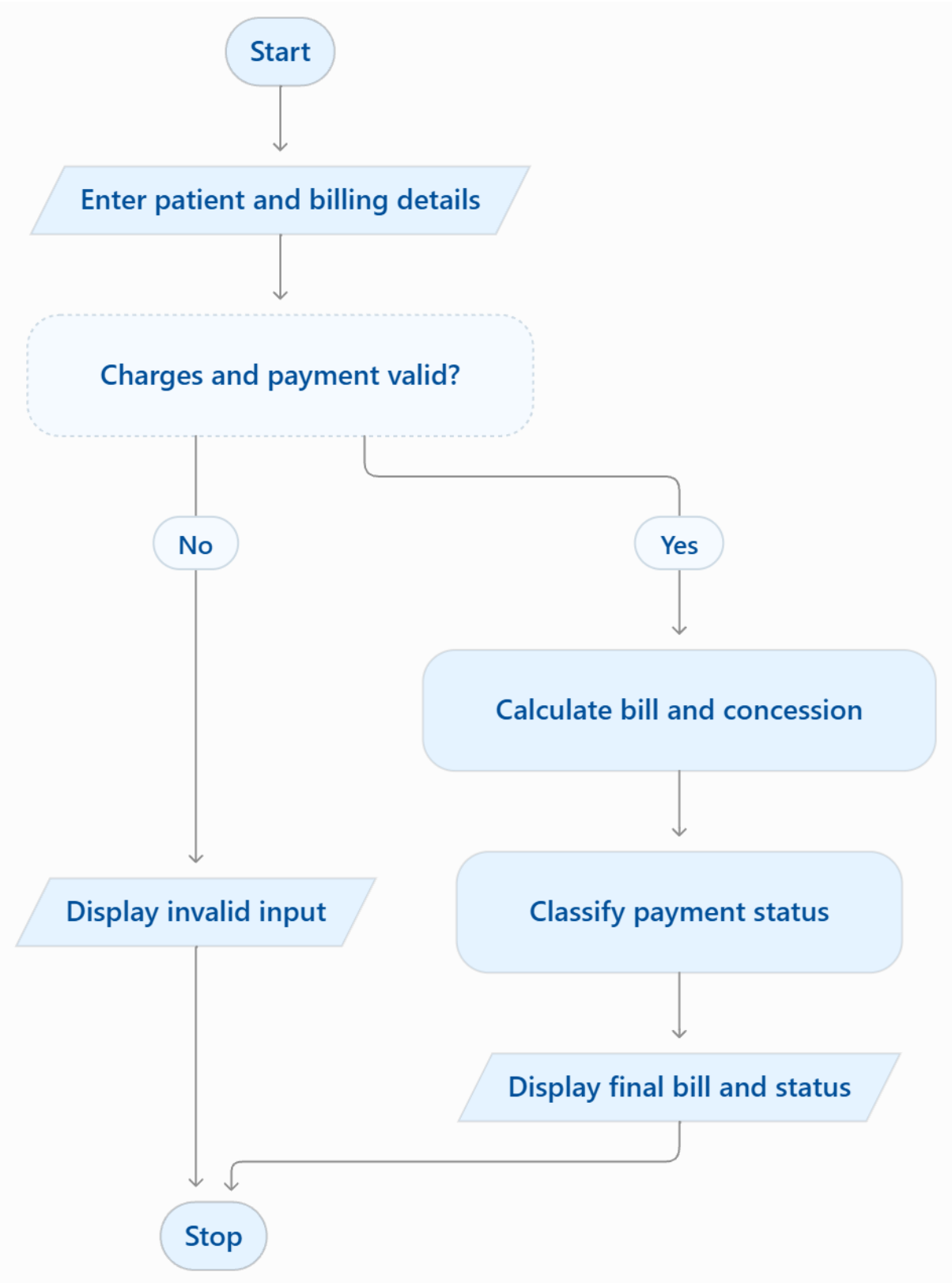
Hospital Billing and Patient Classification: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy

2. Algorithm

1. Start.
2. Input patient name, age, and payment amount.
3. Input consultation, diagnostic, and treatment charges.
4. Validate that charges and payment are non-negative.
5. Calculate the total bill.
6. Apply a 10% concession for senior citizens aged 60 or above.
7. Calculate the final bill.
8. Classify the patient's payment status.
9. Display the bill and payment classification.
10. Stop.

3. Flowchart


Start
Enter patient and billing details
Charges and payment valid?
No
Yes
Display invalid input
Calculate bill and concession
Classify payment status
Display final bill and status
Stop



4.source code

```
name = input("Patient name: ")
```

```
age = int(input("Age: "))
c = float(input("Consultation charge: "))
d = float(input("Diagnostic charge: "))
t = float(input("Treatment charge: "))
p = float(input("Payment amount: "))

if min(age, c, d, t, p) < 0:
    print("Invalid input")
else:
    total = c + d + t
    discount = total * 0.10 if age >= 60 else 0
    bill = total - discount

    if p > bill:
        print("Invalid payment: exceeds bill")
    else:
        status = "Paid" if p == bill else "Partially Paid" if p > 0 else "Unpaid"
        print("Total bill: ₹", total)
        print("Concession: ₹", discount)
        print("Final bill: ₹", bill)
        print("Balance: ₹", bill - p)
        print("Status:", status)
```

5. Sample Input

```
Plain text

Patient name: Ravi
Age: 65
Consultation charge: 500
Diagnostic charge: 1000
Treatment charge: 3000
Payment amount: 2000
```

6. Sample Output

```
Total bill: ₹ 4500.0
Concession: ₹ 450.0
Final bill: ₹ 4050.0
Balance: ₹ 2050.0
Status: Partially Paid
```

Test case 4

Employee Payroll and Deduction Analysis in Python

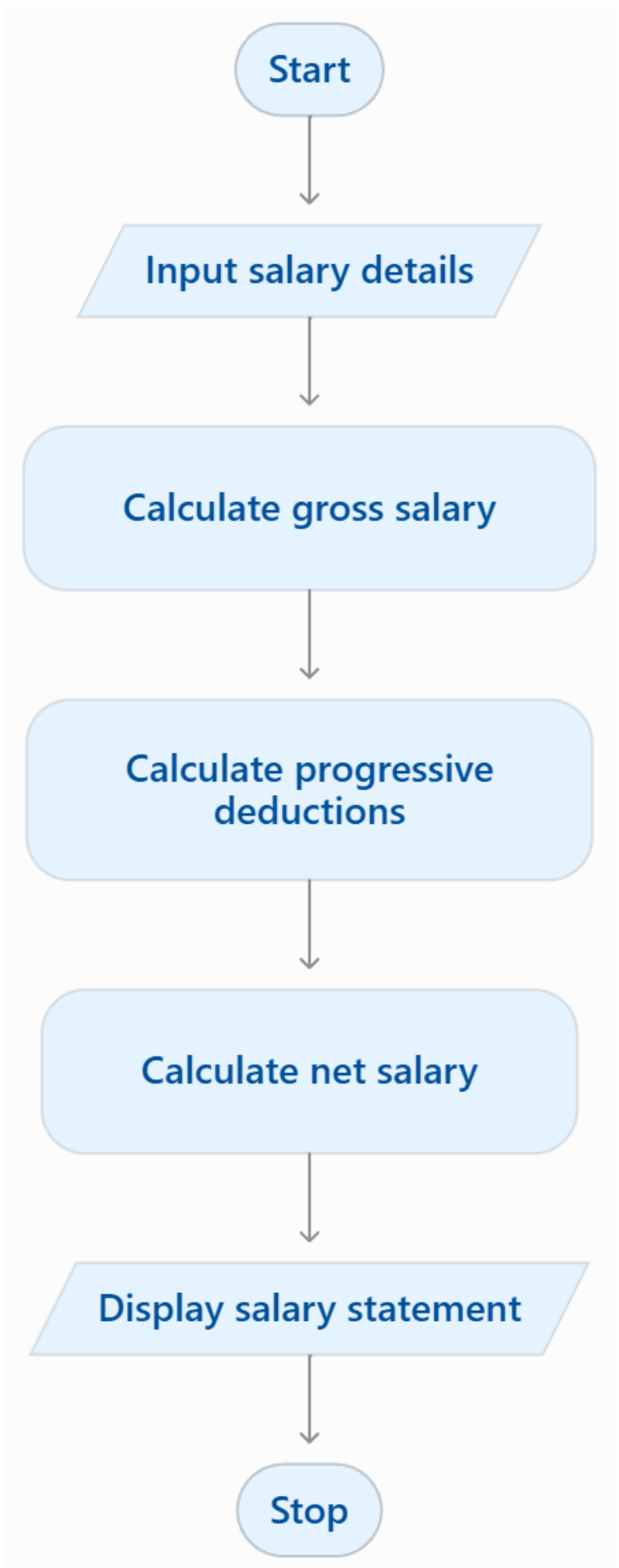
1. Problem Statement

Employee Payroll and Deduction Analysis: Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Algorithm

1. Start.
2. Input employee name, basic pay, allowances, overtime, and incentives.
3. Calculate gross salary.
4. Calculate progressive deductions using salary slabs.
5. Calculate net salary by subtracting deductions from gross salary.
6. Display the formatted salary statement.
7. Stop.

3. Flowchart


Start
Input salary details
Calculate gross salary
Calculate progressive deductions
Calculate net salary
Display salary statement
Stop

4.source code

```
name = input("Employee name: ")
basic = float(input("Basic pay: ₹"))
allow = float(input("Allowances: ₹"))
overtime = float(input("Overtime: ₹"))
incentive = float(input("Incentives: ₹"))

if min(basic, allow, overtime, incentive) < 0:
    print("Invalid salary details")
else:
    gross = basic + allow + overtime + incentive

    if gross <= 20000:
        deduction = gross * 0.05
    elif gross <= 40000:
        deduction = 1000 + (gross - 20000) * 0.10
    else:
        deduction = 3000 + (gross - 40000) * 0.15

    net = gross - deduction
    print("\n--- SALARY STATEMENT ---")
    print("Employee:", name)
    print(f"Gross Salary: ₹{gross:.2f}")
    print(f"Deduction: ₹{deduction:.2f}")
    print(f"Net Salary: ₹{net:.2f}")
```

5. Sample Input

```
Employee name: Ravi
Basic pay: ₹30000
Allowances: ₹5000
Overtime: ₹2000
Incentives: ₹3000
```

6. Sample Output

```
--- SALARY STATEMENT ---
Employee: Ravi
Gross Salary: ₹40000.00
Deduction: ₹3000.00
Net Salary: ₹37000.00
```

Test case 5

Multi-Plan Telecommunications Cost Optimization in Python

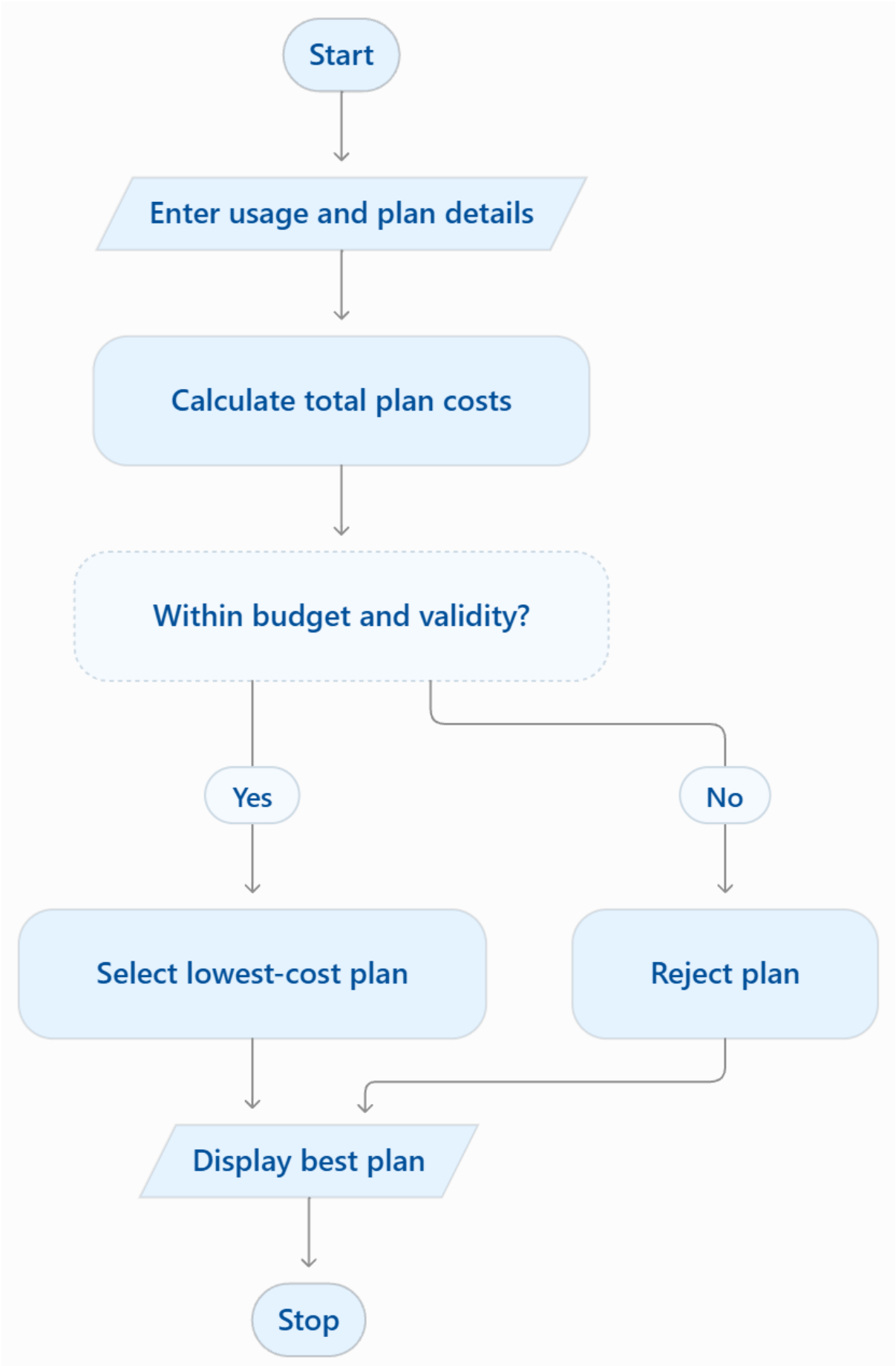
1. Problem Statement

Multi-Plan Telecommunications Cost Optimization: Evaluate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan.

2. Algorithm

1. Start.
2. Input required data usage, call minutes, validity, and budget.
3. Enter the details of multiple mobile plans.
4. Calculate excess data and call charges for each plan.
5. Calculate each plan's total cost.
6. Check validity and budget constraints.
7. Select the lowest-cost eligible plan.
8. Display the plan costs and the best plan.
9. If no plan qualifies, display a suitable message.
10. Stop.

3. Flowchart


Start
Enter usage and plan details
Calculate total plan costs
Within budget and validity?
Yes
No
Select lowest-cost plan
Reject plan
Display best plan
Stop

4.source code

```
data = float(input("Required data (GB): "))
calls = int(input("Required call minutes: "))
days = int(input("Required validity (days): "))
budget = float(input("Budget: ₹"))
n = int(input("Number of plans: "))
plans = []

for i in range(n):
    print(f"\nPlan {i+1}")
    name = input("Plan name: ")
    price = float(input("Base price: ₹"))
    gb = float(input("Included data (GB): "))
    mins = int(input("Included call minutes: "))
    valid = int(input("Validity (days): "))
    cost = price + max(0, data-gb)*10 + max(0, calls-mins)
    ok = valid >= days and cost <= budget
    print(f"Total cost: ₹{cost:.2f}", "Eligible" if ok else "Not eligible")
    if ok:
        plans.append((cost, name))

if plans:
    cost, name = min(plans)
    print(f"\nBest plan: {name}, Cost: ₹{cost:.2f}")
else:
    print("\nNo eligible plan found.")
```

5. Sample Input

```
Required data (GB): 3
Required call minutes: 200
Required validity (days): 28
Budget: ₹300
Number of plans: 3

Plan 1
Plan name: Basic
Base price: ₹199
Included data (GB): 2
Included call minutes: 100
Validity (days): 28

Plan 2
Plan name: Smart
Base price: ₹249
Included data (GB): 3
Included call minutes: 250
Validity (days): 28
```

```
Plan 3
Plan name: Value
Base price: ₹299
Included data (GB): 5
Included call minutes: 300
Validity (days): 56
```

6. Sample Output

```
otal cost: ₹309.00 Not eligible
otal cost: ₹249.00 Eligible
otal cost: ₹299.00 Eligible

est plan: Smart, Cost: ₹249.00
```

TEST case 6

Railway Fare and Reservation Calculator in Python

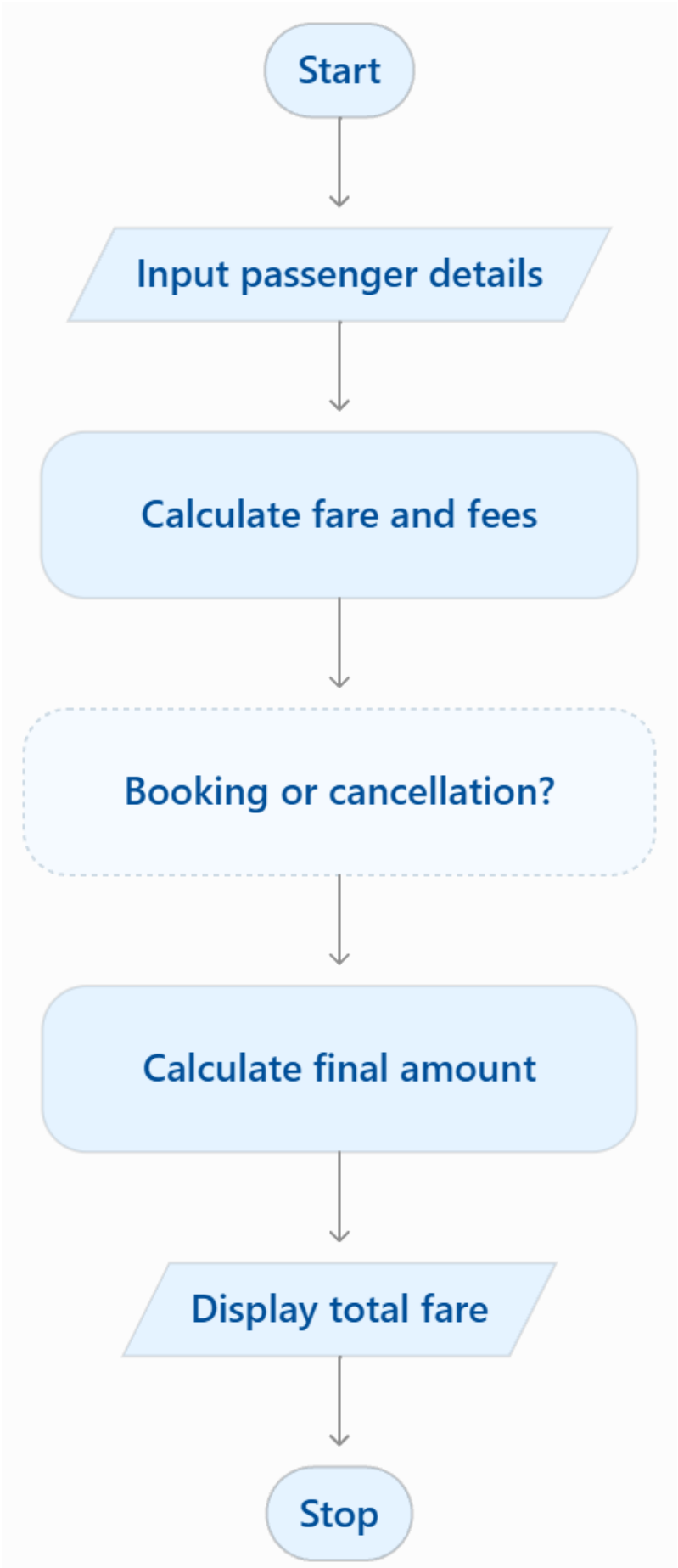
1. Problem Statement

Railway Fare and Reservation Calculator: Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

2. Algorithm

1. Start.
2. Input the number of passengers and travel distance.
3. For each passenger, input name, age, travel class, and booking or cancellation choice.
4. Calculate the base fare using distance slabs.
5. Apply the travel class multiplier and age concession.
6. Add reservation fees for bookings or apply cancellation penalties for cancellations.
7. Display each passenger's fare.
8. Calculate and display the total amount.
9. Stop.

3. Flowchart


Start
Input passenger details
Calculate fare and fees
Booking or cancellation?
Calculate final amount
Display total fare
Stop

4.source code

```
        total = fare * 0.25
        penalty_total += total n = int(input("Number of passengers: "))
distance = int(input("Travel distance (km): "))
booking_total = penalty_total = 0

for i in range(n):
    print(f"\nPassenger {i+1}")
    name = input("Name: ")
    age = int(input("Age: "))
    cls = input("Class (Sleeper/AC): ").lower()
    action = input("Action (Book/Cancel): ").lower()

    if distance <= 100:
        fare = distance * 2
    elif distance <= 300:
        fare = 200 + (distance - 100) * 3
    else:
        fare = 800 + (distance - 300) * 4

    if cls == "ac":
        fare *= 2
    elif cls != "sleeper":
        print("Invalid class!")
        continue

    if age < 5:
        fare = 0
```

```
    elif age >= 60:
        fare *= 0.80

    if action == "book":
        total = fare + 40
        booking_total += total
        print(f"{name}: Booking fare = ₹{total:.2f}")
    elif action == "cancel":

        print(f"{name}: Cancellation penalty = ₹{total:.2f}")
    else:
        print("Invalid action!")

print(f"\nTotal booking amount: ₹{booking_total:.2f}")
print(f"Total cancellation penalties: ₹{penalty_total:.2f}")
```

5. Sample Input

```
Number of passengers: 4
Travel distance (km): 25

Passenger 1
Name: Ravi
Age: 65
Class (Sleeper/AC): Slee
Action (Book/Cancel): Bo

Passenger 2
Name: Priya
Age: 35
Class (Sleeper/AC): AC
Action (Book/Cancel): Bo

Passenger 3
Name: Aman
Age: 4
```

6. Sample Output

```
Class (Sleeper/AC): Sleeper
Action (Book/Cancel): Book

Passenger 4
Name: Sita
Age: 30
Class (Sleeper/AC): AC
Action (Book/Cancel): Cancel
```

Test case 7

Retail Billing with Multiple Discount Rules in Python

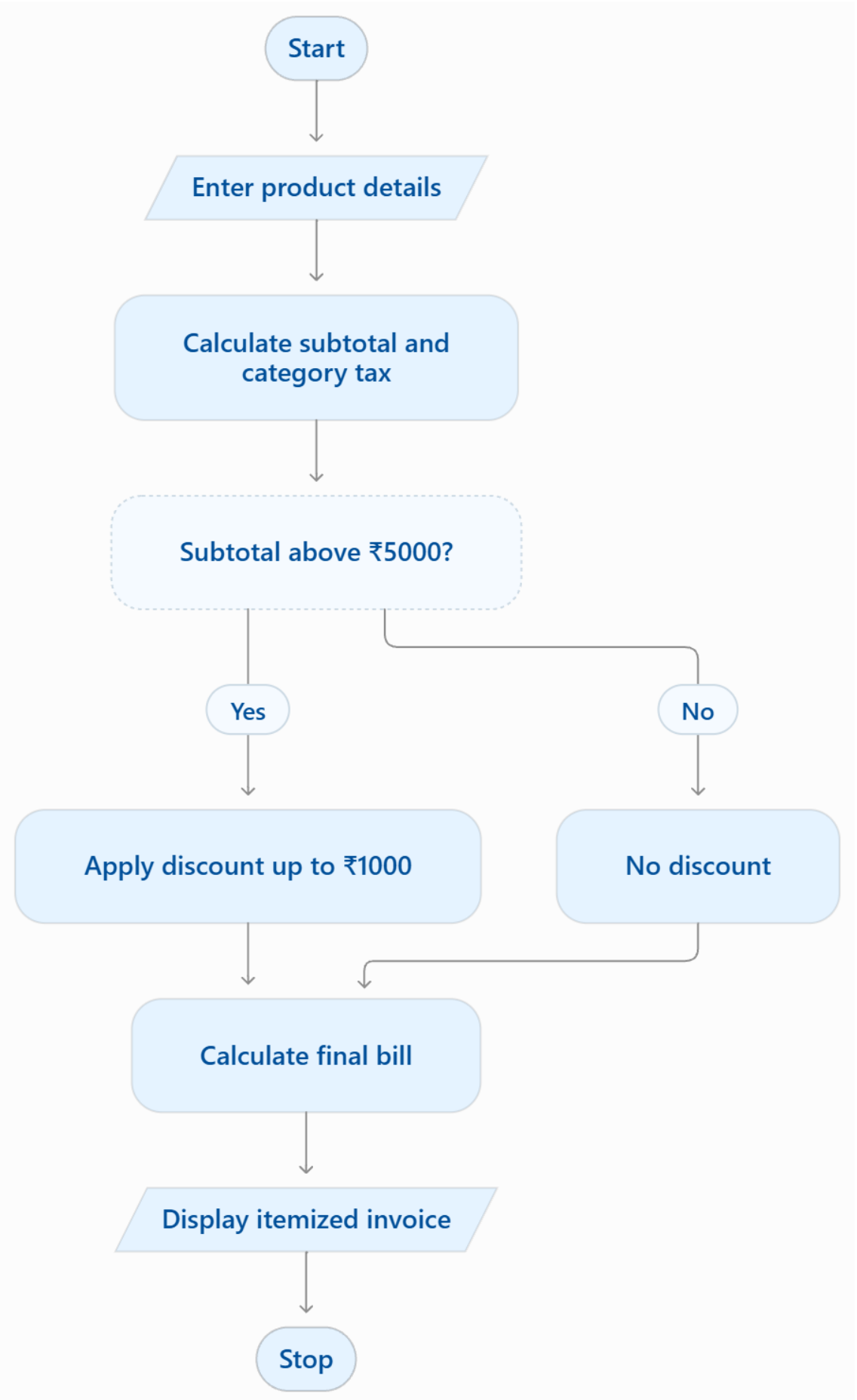
1. Problem Statement

Retail Billing with Multiple Discount Rules: Process multiple products, quantities, categoryspecific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

2. Algorithm

1. Start.
2. Input the number of products.
3. For each product, input name, category, price, and quantity.
4. Calculate each item's subtotal.
5. Apply the discount if the total subtotal exceeds ₹5,000.
6. Limit the discount to ₹1,000.
7. Calculate tax according to each product category.
8. Calculate the final bill.
9. Display the itemized invoice and total amount.
10. Stop.

3. Flowchart


Start
Enter product details
Calculate subtotal and category tax
Subtotal above ₹5000?
Yes
No
Apply discount up to ₹1000
No discount
Calculate final bill
Display itemized invoice
Stop

4.source code

```
n = int(input("Number of products: "))
items = []
subtotal = tax = 0

rates = {"electronics": 0.18, "clothing": 0.05, "other": 0.12}

for i in range(n):
    name = input("Product name: ")
    category = input("Category (electronics/clothing/other): ").lower()
    price = float(input("Unit price: ₹"))
    qty = int(input("Quantity: "))

    if category not in rates or price < 0 or qty <= 0:
        print("Invalid details!")
        continue

    amount = price * qty
    itemtax = amount * rates[category]
    items.append((name, qty, amount, itemtax))
    subtotal += amount
    tax += itemtax

discount = min(subtotal * 0.10, 1000) if subtotal > 5000 else 0

print("\n--- ITEMIZED INVOICE ---")
for name, qty, amount, itemtax in items:
    print(f"{name} x{qty}: ₹{amount:.2f}, Tax: ₹{itemtax:.2f}")
```

print(f"Subtotal: ₹{subtotal:.2f}")

print(f"Discount: ₹{discount:.2f}")

print(f"Total tax: ₹{tax:.2f}")

print(f"Final bill: ₹{subtotal - discount + tax:.2f}")

5. Sample Input

```
Number of products: 2
Product name: Phone
Category (electronics/clothing/other): electronics
Unit price: ₹5000
Quantity: 1
Product name: Shirt
Category (electronics/clothing/other): clothing
Unit price: ₹1000
Quantity: 1
```

6. Sample Output

```
--- ITEMIZED INVOICE ---
Phone x1: ₹5000.00, Tax: ₹900.00
Shirt x1: ₹1000.00, Tax: ₹50.00
Subtotal: ₹6000.00
Discount: ₹600.00
Total tax: ₹950.00
Final bill: ₹6350.00
```

Test Case 8

Manufacturing Cost and Profitability Analysis in Python

1. Problem Statement

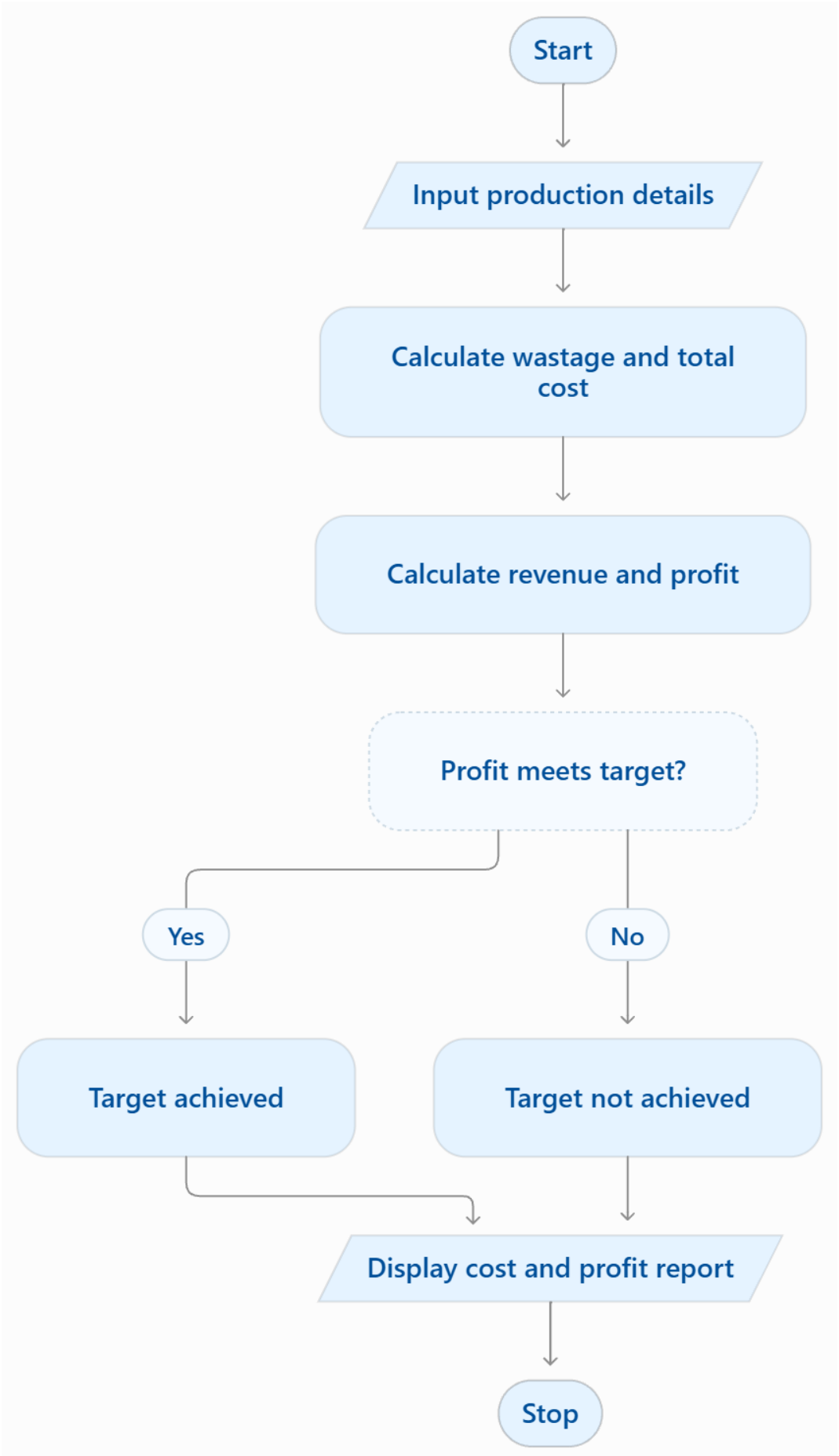
Manufacturing Cost and Profitability Analysis: Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target

2. Algorithm

1. Start.

2. Input material cost, labour charges, machine costs, wastage percentage, quantity produced, selling price per unit, and target profit.
3. Calculate wastage cost.
4. Calculate total manufacturing cost.
5. Calculate total selling revenue.
6. Calculate profit.
7. Compare profit with the target.
8. Display total cost, revenue, profit, and target status.
9. Stop.

3. Flowchart


Start
Input production details
Calculate wastage and total cost
Calculate revenue and profit
Profit meets target?
Yes
No
Target achieved
Target not achieved
Display cost and profit report
Stop

4.source code

```
material = float(input("Material cost: ₹"))
labour = float(input("Labour charges: ₹"))
machine = float(input("Machine cost: ₹"))
waste = float(input("Wastage percentage: "))
qty = int(input("Quantity produced: "))
price = float(input("Selling price per unit: ₹"))
target = float(input("Target profit: ₹"))

if min(material, labour, machine, waste, qty, price, target) < 0 or waste > 100 or qty
== 0:
    print("Invalid input")
else:
    wastage = material * waste / 100
    cost = material + labour + machine + wastage
    revenue = qty * price
    profit = revenue - cost

    print("\n--- PRODUCTION REPORT ---")
    print(f"Wastage cost: ₹{wastage:.2f}")
    print(f"Total cost: ₹{cost:.2f}")
    print(f"Sales revenue: ₹{revenue:.2f}")
    print(f"Profit: ₹{profit:.2f}")
    print("Target achieved" if profit >= target
          else "Target not achieved")
```

5. Sample Input

```
Material cost: ₹10000
Labour charges: ₹4000
Machine cost: ₹2000
Wastage percentage: 5
Quantity produced: 100
Selling price per unit: ₹200
Target profit: ₹3000
```

6. Sample Output

```
--- PRODUCTION REPORT ---
Wastage cost: ₹500.00
Total cost: ₹16500.00
Sales revenue: ₹20000.00
Profit: ₹3500.00
Target achieved
```

Test case 9

Multi-Currency Travel Budget Analysis in Python

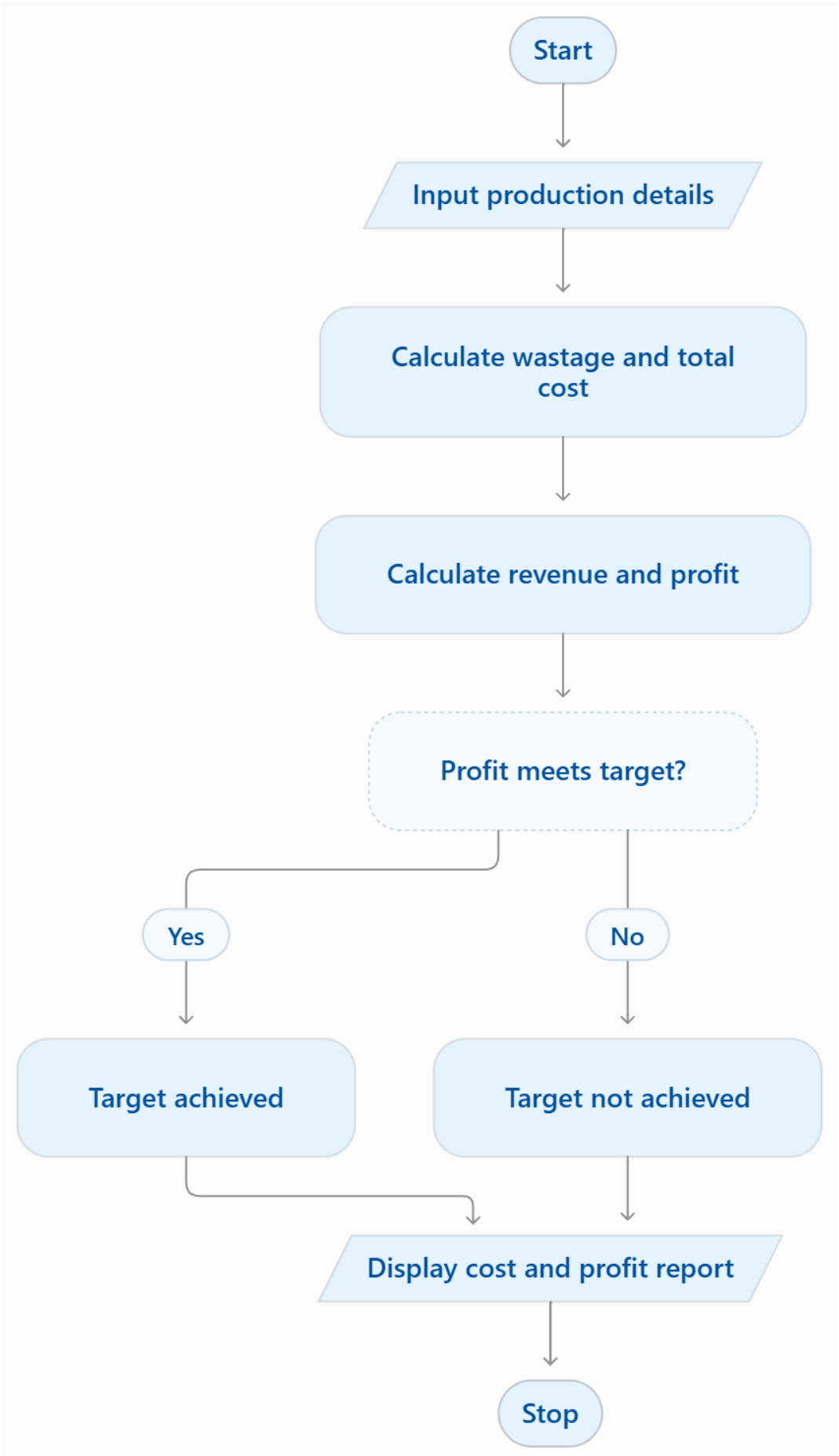
1. Problem Statement

Multi-Currency Travel Budget Analysis: Convert travel expenses using supplied exchange rates, incorporate conversion fees, compare planned spending against a budget, and identify the largest expense category.

2. Algorithm

1. Start.
2. Input the travel budget and conversion fee percentage.
3. Enter expense categories, amounts, currencies, and exchange rates.
4. Convert each expense into INR.
5. Calculate the conversion fee and total expense.
6. Add all expenses to calculate total spending.
7. Identify the largest expense category.
8. Compare spending with the budget.
9. Display the expense report and budget status.
10. Stop.

3. Flowchart


Start
Input production details
Calculate wastage and total cost
Calculate revenue and profit
Profit meets target?
Yes
No
Target achieved
Target not achieved
Display cost and profit report
Stop

4.source code

```
material = float(input("Material cost: ₹"))
labour = float(input("Labour charges: ₹"))
machine = float(input("Machine cost: ₹"))
waste = float(input("Wastage percentage: "))
qty = int(input("Quantity produced: "))
price = float(input("Selling price per unit: ₹"))
target = float(input("Target profit: ₹"))

if min(material, labour, machine, waste, qty, price, target) < 0 or waste > 100 or qty
== 0:
    print("Invalid input")
else:
    wastage = material * waste / 100
    cost = material + labour + machine + wastage
    revenue = qty * price
    profit = revenue - cost

    print("\n--- PRODUCTION REPORT ---")
    print(f"Wastage cost: ₹{wastage:.2f}")
    print(f"Total cost: ₹{cost:.2f}")
    print(f"Sales revenue: ₹{revenue:.2f}")
    print(f"Profit: ₹{profit:.2f}")
    print("Target achieved" if profit >= target
          else "Target not achieved")
```

5. Sample Input

```
Material cost: ₹10000
Labour charges: ₹4000
Machine cost: ₹2000
Wastage percentage: 5
Quantity produced: 100
Selling price per unit: ₹200
Target profit: ₹3000
```

6. Sample Output

```
--- PRODUCTION REPORT ---
Wastage cost: ₹500.00
Total cost: ₹16500.00
Sales revenue: ₹20000.00
Profit: ₹3500.00
Target achieved
```

Test case 10

Campus Event Financial Planning in Python

1. Problem Statement

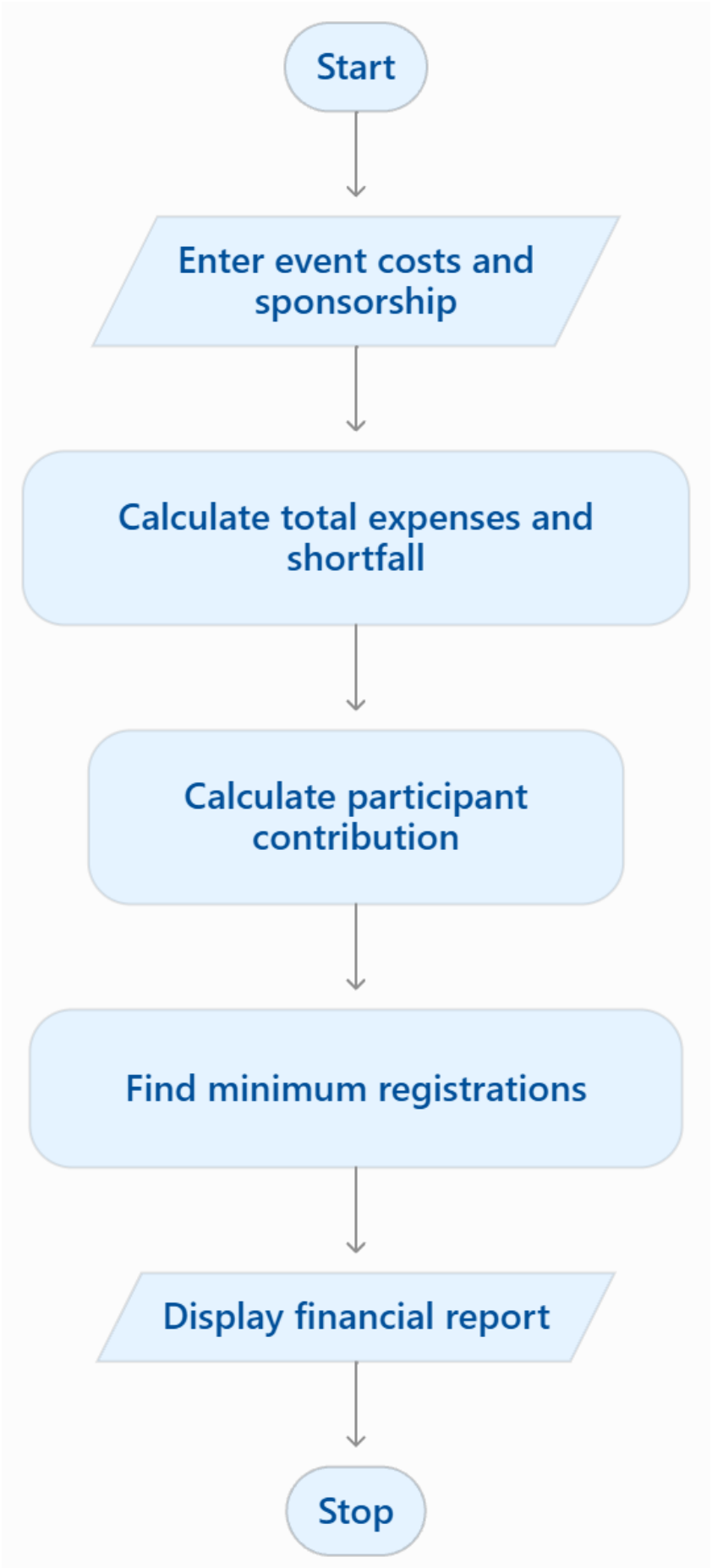
Campus Event Financial Planning: Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

2. Algorithm

1. Start.
2. Input venue, equipment, catering, transport, and certificate costs.
3. Input sponsorship amount, expected participants, and contribution per participant.
4. Calculate total event expenses.
5. Calculate the remaining cost after sponsorship.
6. Calculate the required contribution per participant.
7. Calculate the minimum registrations needed to cover the remaining cost.
8. Display expenses, sponsorship shortfall, participant contribution, and break-even registrations.

9. Stop.

3. Flowchart


Start
Enter event costs and sponsorship
Calculate total expenses and shortfall
Calculate participant contribution
Find minimum registrations
Display financial report
Stop

4.source code

```
venue = float(input("Venue cost: ₹"))
equipment = float(input("Equipment cost: ₹"))
catering = float(input("Catering cost: ₹"))
transport = float(input("Transport cost: ₹"))
certificates = float(input("Certificate cost: ₹"))
sponsor = float(input("Sponsorship amount: ₹"))
participants = int(input("Expected participants: "))
contribution = float(input("Contribution per participant: ₹"))

cost = venue + equipment + catering + transport + certificates
shortfall = max(0, cost - sponsor)

if min(venue, equipment, catering, transport, certificates, sponsor, contribution) < 0 or
participants <= 0:
    print("Invalid input")
else:
    required = shortfall / participants
    registrations = 0 if shortfall == 0 else (
        -(-shortfall // contribution) if contribution > 0 else -1
    )

    print("\n--- EVENT BUDGET REPORT ---")
    print(f"Total cost: ₹{cost:.2f}")
    print(f"Sponsorship shortfall: ₹{shortfall:.2f}")
    print(f"Required contribution per participant: ₹{required:.2f}")
    if registrations >= 0:
        print(f"Minimum registrations to break even: {int(registrations)}")
```

```
    else:
        print("Break-even impossible with zero contribution")
```

5. Sample Input

```
Venue cost: ₹20000
Equipment cost: ₹8000
Catering cost: ₹15000
Transport cost: ₹5000
Certificate cost: ₹2000
Sponsorship amount: ₹20000
Expected participants: 100
Contribution per participant: ₹250
```

6. Sample Output

```
--- EVENT BUDGET REPORT ---
Total cost: ₹50000.00
Sponsorship shortfall: ₹30000.00
Required contribution per participant: ₹300.00
Minimum registrations to break even: 120
```